Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez
Javier Alejandro Penagos Hernandez
Nicolas Castro Rivera

1 Executive Summary

This workshop explores unsupervised learning techniques applied to the KKBox churn prediction domain. A Standard Autoencoder (AE) and a Variational Autoencoder (VAE) are trained on normal customer data and evaluated on their ability to reconstruct inputs and detect churners as anomalies. Results are compared against the supervised baselines from Workshops 1 and 2.

2 Motivation

Workshops 1 and 2 established supervised baselines for churn prediction. This workshop asks whether unsupervised representations can provide complementary signal without access to labels. The AE and VAE are evaluated on two tasks: how accurately they reconstruct customer features, and how well reconstruction error alone separates churners from normal customers. Comparing AUC-ROC across all workshops reveals the gap between supervised and unsupervised approaches in this domain.

3 AE and VAE

Autoencoder and variational autoencoder are unsupervised neural network architectures that learn compressed representations of data without using labels. Both were designed first at the architectural level — defining inputs, hidden layers, bottleneck, and loss function — and then implemented in PyTorch (`torch.nn.Module`).

Standard Autoencoder (AE). The AE was designed as a deterministic encoder–decoder for tabular data: the encoder compresses the input to a fixed-size code vector and the decoder reconstructs the original features from it. Once the design was settled, it was implemented across two modules:

- `autoencoder_ae.py` (`workshops/w4/src/autoencoder/`): defines `AEConfig` (frozen dataclass with all hyperparameters), `Encoder` (stacked linear layers with ReLU ending in a linear projection to z), `Decoder` (mirrored layers branching into a numeric linear head and per-category logit heads), `Autoencoder` (combines both modules with `encode` / `decode` / `forward` methods), and `ReconstructionLoss` (MSE for numeric features plus mean cross-entropy for categorical features).

- `train_ae.py` (`workshops/w4/src/train/`): defines `AETrainer` (training loop with Adam optimiser, per-epoch logging, and best-state tracking), `EarlyStopping` (patience-based stopping on validation loss), `build_loader` (wraps tensors in a `DataLoader`), and checkpoint utilities `save_checkpoint` / `load_checkpoint` that persist the model, config, preprocessor, and training history.

Variational Autoencoder (VAE). The VAE extends the AE with a probabilistic bottleneck: the encoder outputs a mean $\boldsymbol{\mu}$ and a log-variance $\log \boldsymbol{\sigma}^2$ instead of a deterministic code, and a sample $\mathbf{z}$ is drawn via the reparameterization trick during training. The decoder is shared with the AE. A KL divergence term is added to the loss, weighted by β and linearly annealed during the first epochs to avoid posterior collapse. The implementation is split into two modules:

- `autoencoder_vae.py` (`workshops/w4/src/autoencoder/`): defines `VAEConfig` (extends `AEConfig` with `beta`, `kl_anneal_epochs`, and `logvar_clamp`), `VAEEncoder` (shared trunk ending in parallel `mu_head` and `logvar_head`), `VAE` (reparameterization, `sample` method for generation, reuses `Decoder` from the AE module), and `ELBOLoss` (reconstruction summed over feature dimensions plus β-weighted KL divergence).

- `train_vae.py` (`workshops/w4/src/train/`): defines `VAETrainer` (training loop that advances the KL annealer each epoch and tracks active latent units to detect posterior collapse) and `KLAnnealer` (linear warm-up schedule for β). Reuses `EarlyStopping` and `build_loader` from `train_ae.py`; early stopping is delayed until after the KL warm-up phase.

3.1 Data and preprocessing

The autoencoders reuse the tabular export built in Workshop 1 (version 5), where the *members*, *transactions* and *user_logs* tables are condensed into a single record of 59 features per customer (56 numerical and 3 categorical). The February cohort (992,931 customers, 6.39% churn) is used for development and the March cohort (970,960 customers, 8.99% churn) as the held-out test set; 881,701 customers appear in both months.

The Workshop 1 split (stratified and hash-based on `msno`) is reused so that the same customers are used across workshops. The churn label is never fed to the networks; it is only used to select the training population. The AE and the VAE are trained on the 697,095 non-churn customers of the *fit* partition and validated on the 116,183 non-churn customers of the *stop* partition, while the full March cohort, including churners, is used for testing.

A preprocessor is fitted on the training population only. Numerical features are imputed with the median, skewed features are transformed with a signed $\log(1+|x|)$, clipped to the 0.1% and 99.9% quantiles and standardized; one constant column was removed, leaving 55 numerical inputs. The categorical features (`city`, `registered_via`, `last_payment_method_id`) are mapped to vocabularies of 22, 6 and 34 levels and one-hot encoded inside the network, giving an input dimension of 117. An inverse transform maps reconstructions back to the original 59 columns. The AE and the VAE share the same fitted preprocessor.

3.2 AE Architecture

The autoencoder receives tabular input of dimension $d_{\text{in}} = 117$: 55 standardised numerical features concatenated with the one-hot encodings of the three categorical columns (`city`: 22, `registered_via`: 6, `last_payment_method_id`: 34).

Encoder. Two fully-connected layers with decreasing hidden units compress the input to the bottleneck:

$$\mathbb{R}^{117} \xrightarrow{W_1,\,\text{ReLU}} \mathbb{R}^{64} \xrightarrow{W_2,\,\text{ReLU}} \mathbb{R}^{32} \xrightarrow{W_z} \mathbb{R}^{z}.$$

Bottleneck. Three latent dimensions were evaluated: $z \in \{8, 16, 32\}$. A linear projection (no activation) maps the last hidden representation to the code vector $\mathbf{z}$, which is the only information passed to the decoder.

Decoder. The decoder mirrors the encoder with reversed hidden widths $[32, 64]$ and then branches into two output heads:

- *Numeric head*: a linear layer (no activation) reconstructing the 55 standardised numerical features.
- *Categorical heads*: one independent linear layer per categorical feature, producing un-normalised class logits (22, 6, and 34 dimensions respectively).

Loss function. The reconstruction loss combines mean squared error for the numerical part with the mean cross-entropy over all categorical heads, controlled by a scalar weight $\lambda = 1.0$:

$$\mathcal{L} = \underbrace{\text{MSE}(\hat{\mathbf{x}}_{\text{num}},\, \mathbf{x}_{\text{num}})}_{\text{numeric}} + \lambda \underbrace{\frac{1}{C}\sum_{c=1}^{C} \text{CE}(\hat{\mathbf{x}}_c,\, x_c)}_{\text{categorical}}.$$

Table 1 lists the hyperparameters used and the rationale behind each choice.

Table 1: AE hyperparameters and selection rationale.

Hyperparameter	Value	Rationale
`hidden_dims`	(64, 32)	Funnel shape: halves width at each layer from 117 to 64, 32, then z. Two layers suffice for tabular inputs of this size; a common heuristic.
`z_dim`	16	Chosen by experiment (sweep over 8 / 16 / 32): balances reconstruction quality and compression.
`learning_rate`	10^{-3}	Default recommended for Adam [1]. Loss curves decrease smoothly without oscillation, confirming it is adequate.
`batch_size`	512	Gives ≈1,362 updates per epoch (697,095 / 512); consistent with the batch size used in Workshop 2.
`max_epochs`	50	Upper bound set by the workshop guidelines (30–50 epochs).
`patience`	5	Early stopping: training halts if validation loss does not improve for 5 consecutive epochs.
`dropout`	0.0	No dropout needed; train and validation curves track closely, indicating no overfitting.
`weight_decay`	0.0	Same reasoning as dropout; regularisation is not required given the data volume.
`cat_loss_weight`	1.0	Numerical and categorical components contribute equally to the total loss.
`seed`	110	Same seed used in Workshops 1 and 2 for reproducibility across experiments.

Figure 1 illustrates the full encoder–bottleneck–decoder pipeline.

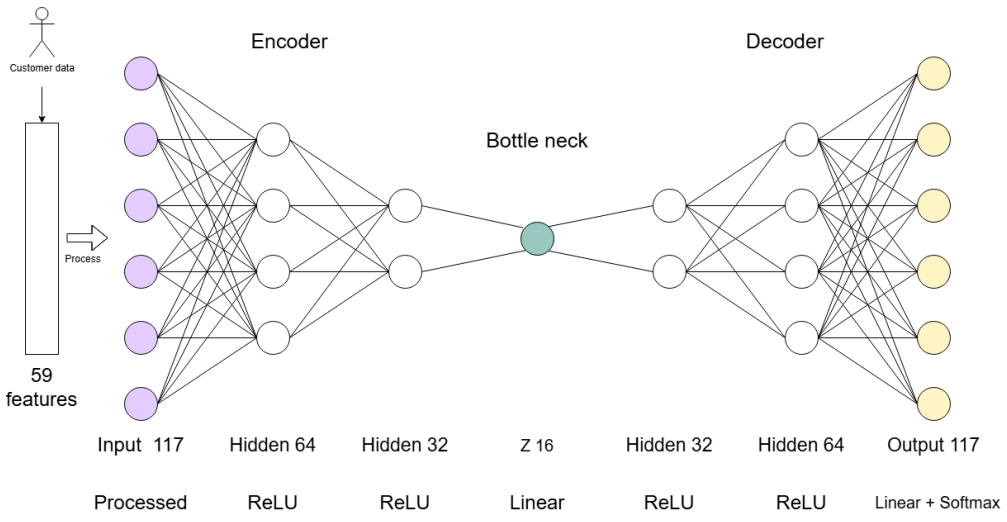


Figure 1: Standard Autoencoder architecture. The encoder compresses the 117-dimensional input to a z-dimensional code; the decoder reconstructs numerical features via a linear head and categorical features via per-class logit heads.

3.3 AE Train

All three latent configurations share the same training protocol. The Adam optimiser [1] is used with learning rate 10^{-3}, no weight decay, and a batch size of 512. Training runs for up to 50 epochs with early stopping (patience 5) monitoring the validation reconstruction loss. The model state corresponding to the lowest validation loss is restored at the end of training.

The training set consists of 697,095 non-churn customers from the February fit partition; the 116,183 non-churn customers of the stop partition are used as the validation set. The churn label is never seen during training.

Table 2 summarises the final train and validation losses for each bottleneck size. None of the three runs triggered early stopping; all completed the full 50 epochs.

Table 2: AE training results by latent dimension (epoch 50).

z	Train loss	Val loss	Train MSE	Val MSE
8	0.1274	0.1274	0.1250	0.1256
16	0.0443	0.0427	0.0433	0.0421
32	0.0161	0.0157	0.0156	0.0152

Reconstruction error drops sharply as z grows: z=32 achieves a validation loss roughly 8× lower than z=8, confirming that larger bottlenecks retain more customer-level information. Figure 2 shows the training and validation loss curves for all three runs.

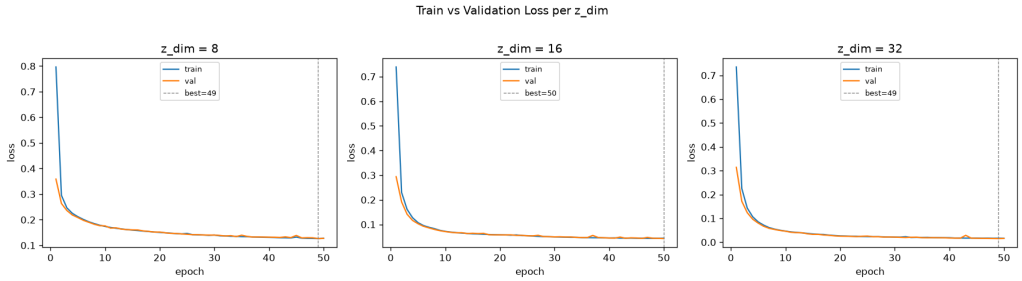


Figure 2: Training and validation reconstruction loss per epoch for $z \in \{8, 16, 32\}$. All curves converge smoothly without overfitting, and larger bottlenecks achieve consistently lower loss.

3.4 VAE Architecture

The VAE shares the same input representation as the AE (117 dimensions: 55 numerical and 62 one-hot). The key difference lies in the encoder and the loss function.

Encoder. A shared trunk identical to the AE encoder processes the input through two hidden layers:

$$\mathbb{R}^{117} \xrightarrow{W_1,\,\mathrm{ReLU}} \mathbb{R}^{64} \xrightarrow{W_2,\,\mathrm{ReLU}} \mathbb{R}^{32}.$$

Two parallel linear heads then project the trunk output to the parameters of the posterior distribution:

$$\boldsymbol{\mu} = W_\mu\,\mathbf{h}, \qquad \log \boldsymbol{\sigma}^2 = \mathrm{clamp}(W_\sigma\,\mathbf{h},\, -10,\, 10).$$

Bottleneck. A sample $\mathbf{z} \in \mathbb{R}^z$ is drawn during training via the reparameterization trick:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

At inference the deterministic mean $\boldsymbol{\mu}$ is used instead. The same three latent dimensions as the AE were evaluated: $z \in \{8, 16, 32\}$; z=16 was selected.

Decoder. The same `Decoder` module from the AE is reused unchanged: mirrored hidden layers $[32, 64]$ with ReLU, a linear numeric head, and per-category logit heads.

Loss (ELBO). The training objective is the Evidence Lower Bound, composed of a reconstruction term and a KL divergence weighted by β:

$$\mathcal{L}_{\mathrm{ELBO}} = \underbrace{\sum_{\text{num}} \mathrm{MSE}_i + \sum_{c=1}^{C} \mathrm{CE}_c}_{\text{reconstruction}} + \beta \underbrace{\left(-\tfrac{1}{2} \sum_{j=1}^{z} \big(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\big) \right)}_{\text{KL divergence}}.$$

β is linearly annealed from 0 to its target value over the first 10 epochs to prevent posterior collapse. Three values were evaluated: $\beta \in \{0.5, 1.0, 4.0\}$.

Table 3 lists the VAE-specific hyperparameters and their rationale.

Table 3: VAE-specific hyperparameters and selection rationale. All other hyperparameters match Table 1.

Hyperparameter	**Value**	**Rationale**
`beta`	1.0	Chosen by experiment (sweep over 0.5 / 1.0 / 4.0): balances reconstruction quality and latent space regularity. Standard β-VAE baseline.
`kl_anneal_epochs`	10	Linear warm-up prevents the KL term from dominating in early epochs, which would collapse the posterior before the decoder learns to reconstruct.
`logvar_clamp`	$(-10, 10)$	Clamps $\log \sigma^2$ to avoid numerical overflow in $\exp(\log \sigma^2)$ during the reparameterization step.

Figure 3 illustrates the full VAE pipeline.

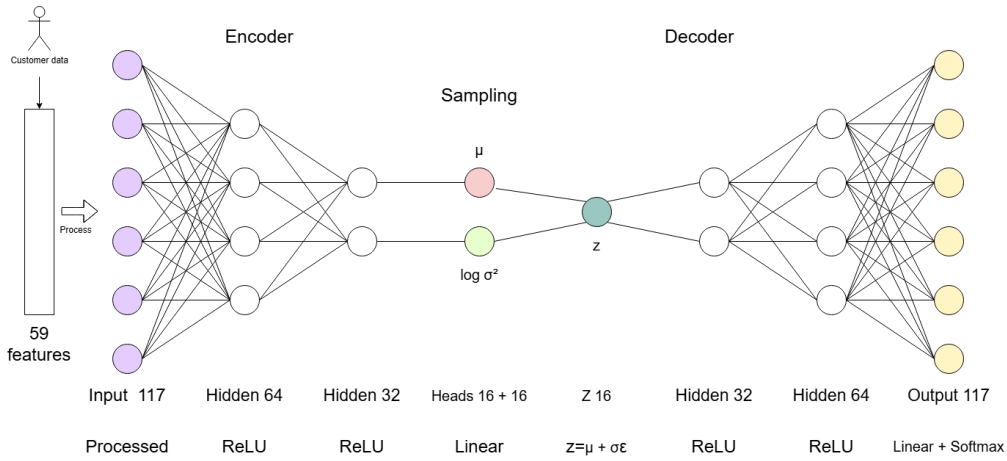


Figure 3: Variational Autoencoder architecture. The encoder trunk branches into $\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$ heads; a sample is drawn via the reparameterization trick and passed to the shared decoder.

3.5 VAE Train

The VAE follows the same training protocol as the AE (Adam, lr $= 10^{-3}$, batch size 512, up to 50 epochs), with two differences: early stopping is delayed until after the KL warm-up phase (epoch > 10), and the KL annealer advances β_t linearly each epoch. Three values of β were evaluated: $\{0.5, 1.0, 4.0\}$.

Unlike the AE runs, all three VAE experiments triggered early stopping before reaching epoch 50. The active latent units metric — the number of dimensions with $\mathrm{Var}(\boldsymbol{\mu}) > 0.01$ across the validation set — is tracked each epoch to monitor posterior collapse.

Table 4 summarises the results at the best checkpoint for each run.

Table 4: VAE training results by β at the best validation epoch.

β	Stop ep.	Best ep.	Val ELBO	Val recon	Val KL	Active units
0.5	26	21	10.575	3.603	13.943	15
1.0	27	22	15.304	5.997	9.308	13
4.0	26	21	28.958	16.355	3.151	6

Higher β penalises the KL term more strongly, forcing the posterior closer to $\mathcal{N}(\mathbf{0}, \mathbf{I})$ at the cost of reconstruction quality: β=4.0 achieves the lowest KL (3.15) but the worst reconstruction (16.36) and collapses 10 of the 16 latent dimensions. β=0.5 retains 15 active units and the best reconstruction (3.60) but with a less regular latent space. β=1.0 was selected as the best trade-off. Figure 4 shows the training and validation ELBO curves for all three runs.

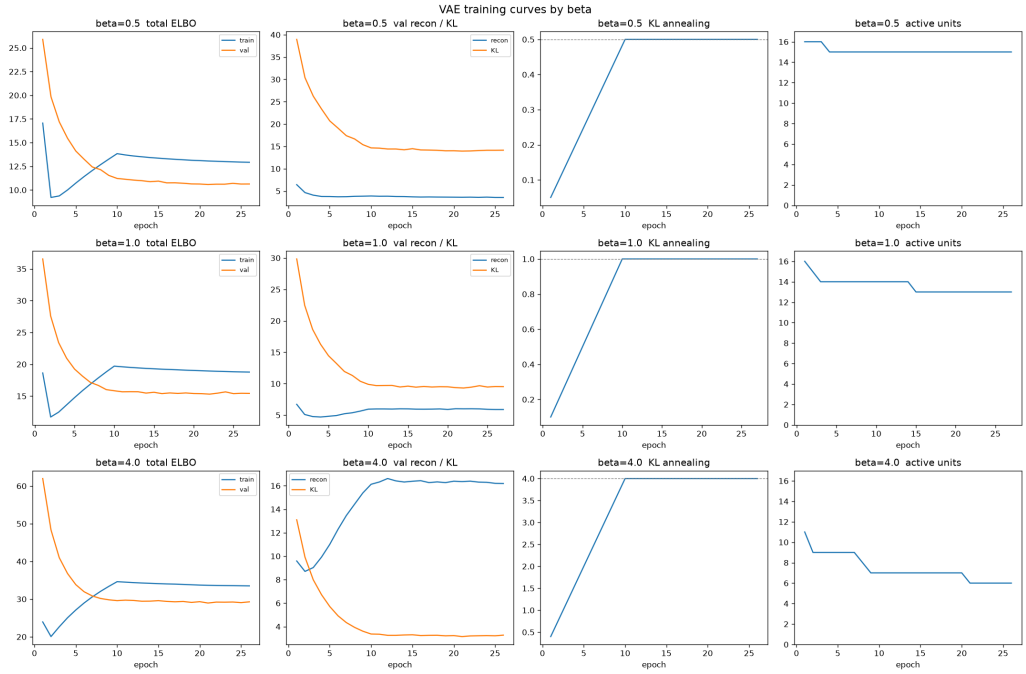


Figure 4: Training and validation ELBO per epoch for $\beta \in \{0.5, 1.0, 4.0\}$. The KL warm-up phase (first 10 epochs) is visible as a rising loss before convergence. All runs triggered early stopping before epoch 50.

4 Reconstruction quality

Both the AE and the best VAE (z=16, β=1.0) were evaluated on the full March test cohort (970,960 customers, 8.99% churn). The churn label was not used during training and is only used here to stratify the results.

4.1 Test set metrics

Table 5 reports global and class-stratified metrics. For continuous features, mean Pearson correlation r measures how well the reconstructed values track the originals across the test set. For categorical features, top-1 accuracy is reported against a most-frequent baseline.

Table 5: Reconstruction quality on the March test set. Categorical accuracy baseline uses the most-frequent class per feature.

Metric	**AE**	**VAE**
MSE global	0.0519	0.0943
MSE normal customers	0.0417	0.0834
MSE churn customers	0.1543	0.2052
Churn / normal MSE ratio	3.70	2.46
Mean Pearson r (numeric)	0.9764	0.9614
Acc. `city`	99.98%	66.03%
Acc. `registered_via`	99.99%	89.44%
Acc. `last_payment_method_id`	99.92%	86.71%
Baseline `city` (most-frequent)	45.58%	
Baseline `registered_via`	58.98%	
Baseline `last_payment_method_id`	56.08%	

The AE achieves a 1.8× lower global MSE than the VAE and near-perfect categorical accuracy, reflecting its deterministic bottleneck which preserves more input detail. The VAE's probabilistic latent space sacrifices some reconstruction fidelity for a more regularised representation. Both models maintain high Pearson correlations ($r > 0.96$), indicating that the overall structure of continuous features is well preserved.

4.2 Sample reconstructions

Figure 5 shows eight test customers (four normal, four churners) reconstructed by both models across seven representative features. The colour scale maps standardised feature values, allowing direct visual comparison of original, AE, and VAE outputs.

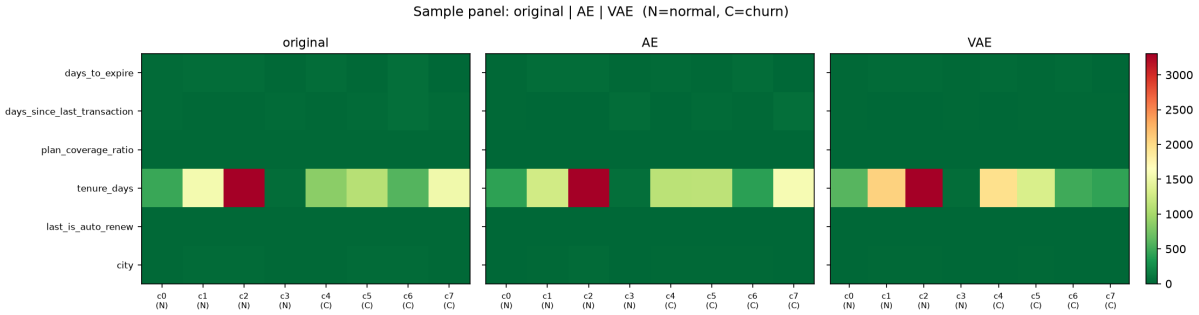


Figure 5: Sample panel for 8 test customers (N=normal, C=churn) across 7 features. Columns show original values, AE reconstruction, and VAE reconstruction. Colour scale is shared across all three panels.

Figure 6 presents element-wise absolute difference heatmaps $|\mathbf{x} - \hat{\mathbf{x}}|$ for the same 8 customers over the 20 hardest features. Both maps use the same colour scale to allow direct comparison.

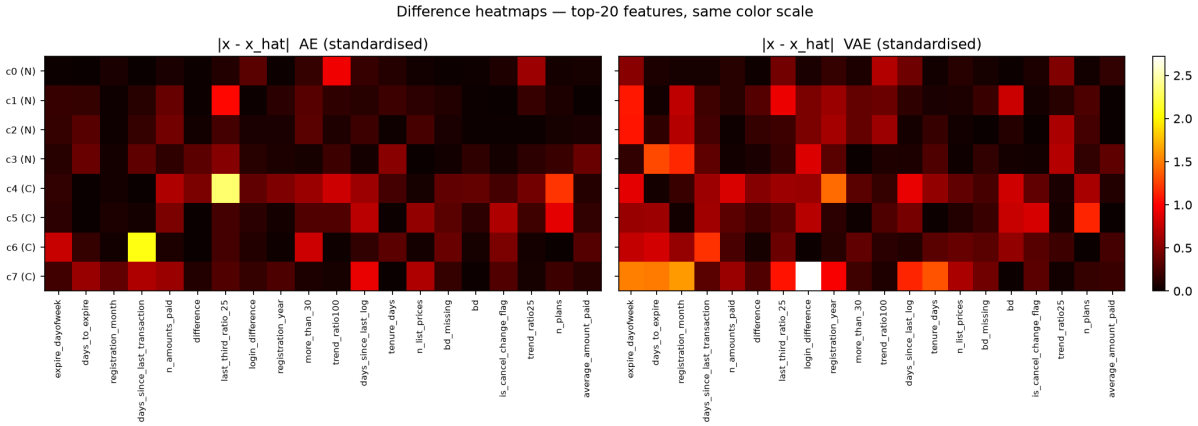


Figure 6: Absolute difference heatmaps for AE and VAE on 8 sample customers (same colour scale). Brighter cells indicate larger reconstruction error. Churn customers (C) tend to show wider error spread.

4.3 Error patterns and analysis

Churn customers are harder to reconstruct. Both models were trained exclusively on normal customers, so churn customers represent out-of-distribution inputs. This is reflected in the MSE ratios: the AE makes 3.70× larger errors on churners than on normal customers; the VAE 2.46×. The lower ratio for the VAE is not a sign of better churn modelling — its overall MSE is higher — but a consequence of the regularised prior smoothing all reconstructions toward the mean. Figure 7 shows the per-sample MSE distributions stratified by class for both models.



Figure 7: Per-sample MSE distribution (log scale) for normal and churn customers. Both models produce noticeably higher errors on churners, confirming that churn behaviour is outside the manifold learned during training.

Figure 8 shows the full MSE density, confirming that the normal-vs-churn separation is consistent across the test set.

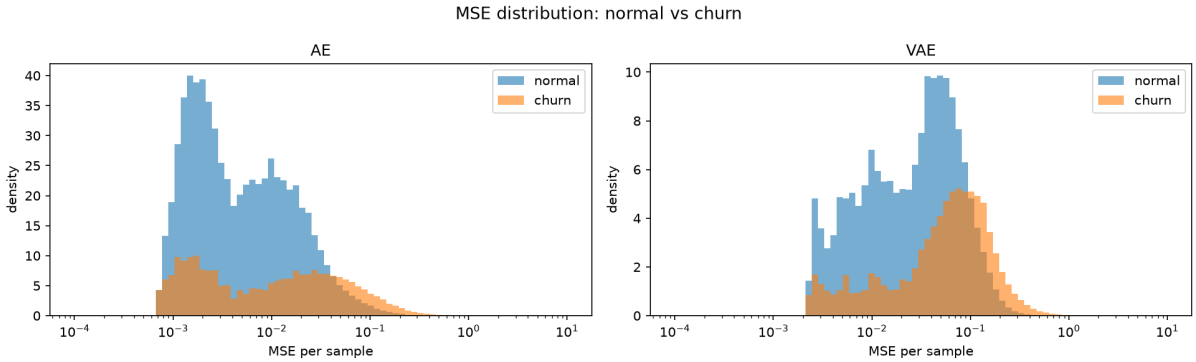


Figure 8: MSE density for normal and churn customers (log-scale x-axis). The churn distribution is shifted to the right in both models, with a longer tail indicating a subset of churners with very high error.

Temporal and cyclical features are hardest to reconstruct. Table 6 lists the five features with the highest VAE MSE. Temporal features — `expire_dayofweek`, `days_to_expire`, `registration_month` — dominate the top positions. These features have irregular or multimodal distributions tied to billing cycles, which are difficult for the bottleneck to compress faithfully. The AE handles most of them better than the VAE, with the starkest gap on `registration_month` (ratio 17.8) and `expire_dayofweek` (ratio 5.3). Figure 9 shows the top-15 hardest features for both models.

Table 6: Top-5 features with highest VAE MSE on the test set.

Feature	MSE AE	MSE VAE	VAE/AE
`expire_dayofweek`	0.067	0.351	5.3×
`days_to_expire`	0.157	0.267	1.7×
`registration_month`	0.014	0.241	17.8×
`days_since_last_transaction`	0.179	0.196	1.1×
`n_amounts_paid`	0.083	0.181	2.2×

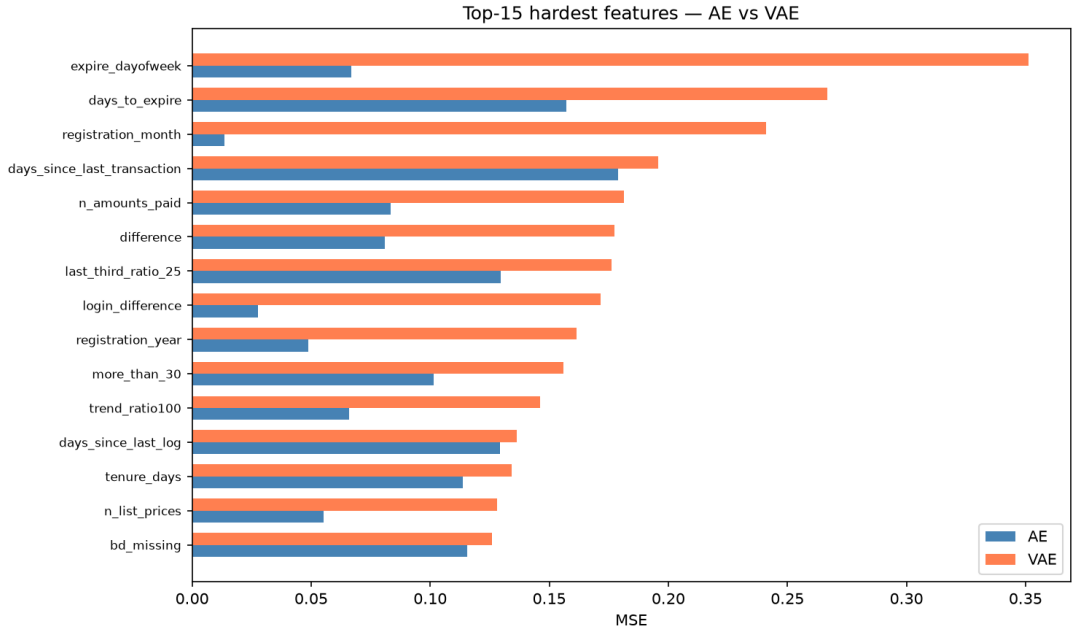


Figure 9: Top-15 features with highest MSE for AE and VAE. Temporal and cyclical features consistently appear at the top; the VAE shows larger errors on most of them.

5 Anomaly detection

Churn customers are treated as anomalies: both models were trained exclusively on normal customers and are expected to reconstruct churners poorly. A per-customer anomaly score is derived from the reconstruction error; customers with higher scores are flagged as anomalous. No label information is used beyond selecting the training population.

5.1 Implementation

The anomaly detection pipeline is implemented in `anomaly_detection.py` (`workshops/w4/src/evaluate/`):

- `AnomalyScorer`: computes three anomaly scores in a single forward pass over the test set using batches of 4,096 samples.
  - `AE_recon`: per-sample reconstruction error from the AE — sum of squared errors (numeric) plus sum of cross-entropies (categorical).
  - `VAE_recon`: same reconstruction error computed from the VAE.
  - `VAE_recon_kl`: VAE reconstruction error plus the per-sample KL divergence $D_{\mathrm{KL}}(q(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I}))$, giving a full ELBO-based anomaly score.
- `ThresholdSelector`: selects the decision threshold that maximises churn F1 on the tune set, keeping the test set completely held out.
- `AnomalyEvaluator`: reports ranking metrics (AUC-ROC, average precision) and decision metrics (precision, recall, F1, balanced accuracy, MCC) at the selected threshold.

- `CumulativeResults`: merges W4 results with the supervised baselines from Workshops 1 and 2 for a unified comparison table.

5.2 Score distributions and ranking metrics

Figure 10 shows the anomaly score distributions for normal and churn customers on the test set. All three scores separate the classes, with the churn distribution shifted toward higher values as expected. The red dashed line marks the threshold selected on the tune set.

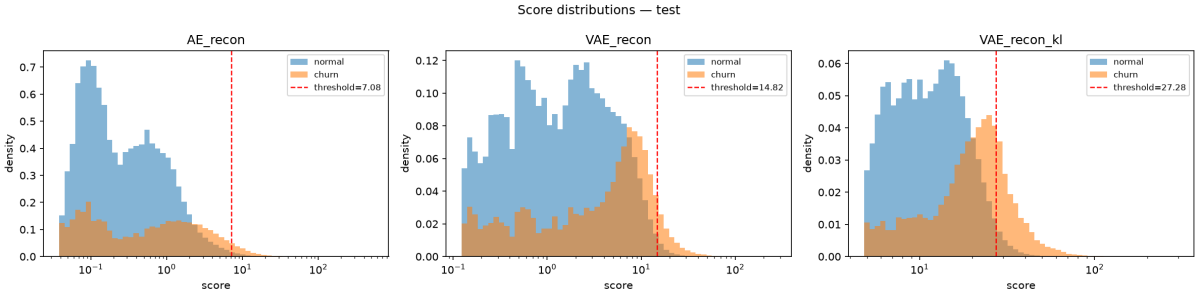


Figure 10: Anomaly score distributions (log scale) for normal and churn customers. The red dashed line is the threshold maximising churn F1 on the tune set. `VAE_recon_kl` shows the strongest class separation.

Figures 11 and 12 show the ROC and precision-recall curves. `VAE_recon_kl` dominates both curves, confirming that adding the KL term provides a meaningful improvement over reconstruction error alone.

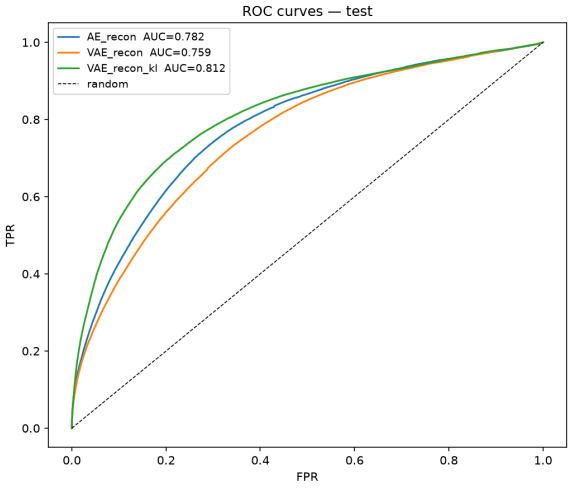


Figure 11: ROC curves on the test set. `VAE_recon_kl` achieves AUC = 0.812.

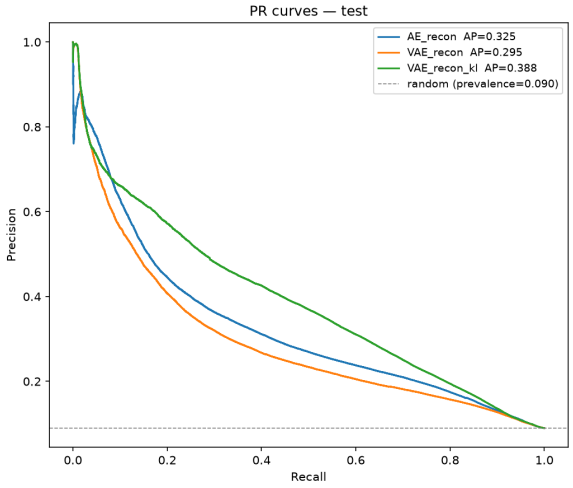


Figure 12: PR curves on the test set. Random baseline at prevalence = 0.090.

5.3 Decision metrics at selected threshold

Table 7 reports full decision metrics at the tune-set threshold for each score. The confusion matrix for the best scorer (`VAE_recon_kl`) is shown in Figure 13.

Table 7: Anomaly detection metrics on the test set (threshold selected on tune set). Test prevalence: 8.99%.

Score	AUC-ROC	AP	Thresh.	Prec.	Recall	F1	Bal. Acc.
`VAE_recon_kl`	**0.8116**	**0.3882**	27.28	0.417	0.413	**0.415**	0.678
`AE_recon`	0.7823	0.3248	7.08	0.346	0.333	0.339	0.635
`VAE_recon`	0.7589	0.2947	14.82	0.351	0.256	0.296	0.605

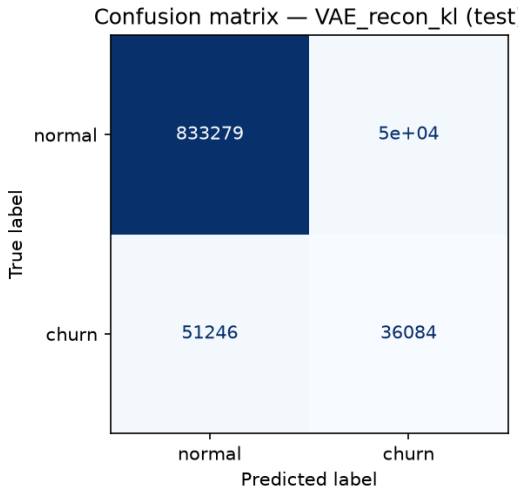


Figure 13: Confusion matrix for `VAE_recon_kl` on the test set (threshold = 27.28, selected on tune set).

5.4 Comparison with W1 and W2 baselines

Table 8 presents all models from Workshops 1, 2, and 4 ranked by AUC-ROC. The supervised models retain a clear advantage, benefiting from direct access to churn labels. `VAE_recon_kl` is the best unsupervised detector and closes most of the gap to supervised approaches in AUC (0.812 vs. 0.863 for the best supervised model), while showing substantially lower precision and recall due to the harder task of detecting anomalies without label supervision.

Table 8: Cumulative model comparison across Workshops 1, 2, and 4, sorted by AUC-ROC.

Model	Family	WS	Type	AUC-ROC	AP	F1	Recall
M2	Random Forest	W1	supervised	0.863	0.593	0.584	0.539
M3	XGBoost	W1	supervised	0.852	0.588	0.574	0.542
LSTM	LSTM	W2	supervised	0.851	0.580	0.578	0.518
M1	Logistic	W1	supervised	0.844	0.555	0.557	0.534
`VAE_recon_kl`	Autoencoder	W4	unsupervised	**0.812**	0.388	0.415	0.413
`AE_recon`	Autoencoder	W4	unsupervised	0.782	0.325	0.339	0.333
`VAE_recon`	Autoencoder	W4	unsupervised	0.759	0.295	0.296	0.256

The VAE ELBO score (`VAE_recon_kl`) surpasses the AE by 2.9 AUC points, confirming that the KL term — which measures how far a customer's posterior deviates from the learned prior — adds orthogonal signal beyond reconstruction error alone. Pure `VAE_recon` underperforms the AE, reflecting the reconstruction quality trade-off already observed in Section 3: the regularised VAE bottleneck reconstructs all inputs somewhat uniformly, reducing the signal-to-noise ratio of reconstruction error as an anomaly detector.

6 Latent space visualization

The latent codes learned by the AE and VAE encoders are analysed to understand how the two architectures organise the customer manifold and whether the representations separate churners from normal customers without ever seeing labels.

6.1 Implementation

The analysis is split across two modules in `workshops/w4/src/analize_latent/`:

- `analyze_latent.py`: three classes support the full pipeline. `LatentExtractor` encodes the test set in batches of 2,048 — using `encode()` for the AE and the deterministic mean $\boldsymbol{\mu}$ (not a sampled $\mathbf{z}$) for the VAE. `LatentProjector` reduces representations to 2D via t-SNE (perplexity = 30, PCA init) and UMAP (n_neighbors = 15, min_dist = 0.1). `SeparabilityAnalyzer` quantifies class separation with the Silhouette score (raw and after StandardScaler) and a linear probe — a logistic regression evaluated with 5-fold stratified cross-validation whose mean AUC measures how linearly separable the two classes are in the latent space.
- `extract_lstm_latent.py`: a subprocess script that extracts the hidden states from the Workshop 2 LSTM encoder for direct comparison. The W2 sequential export was not available in the current environment, so the LSTM comparison is omitted from the quantitative results; the analysis covers AE and VAE only.

Two samples are used throughout: a 20,000-point stratified sample (natural prevalence ≈9% churn) for separability metrics, and a balanced 10,000-point sample (5,000 normal + 5,000 churn) for 2D projections to avoid the visualisation being dominated by the majority class.

6.2 Latent dimension analysis

Figure 14 shows the standard deviation per latent dimension for both models. The AE dimensions have varied and generally large standard deviations with no collapsed dimensions. The VAE dimensions cluster near the unit Gaussian prior ($\sigma = 1$), with 3 of the 16 dimensions below the collapse threshold ($\text{Var}(\boldsymbol{\mu}) < 0.01$), leaving 13 active dimensions.

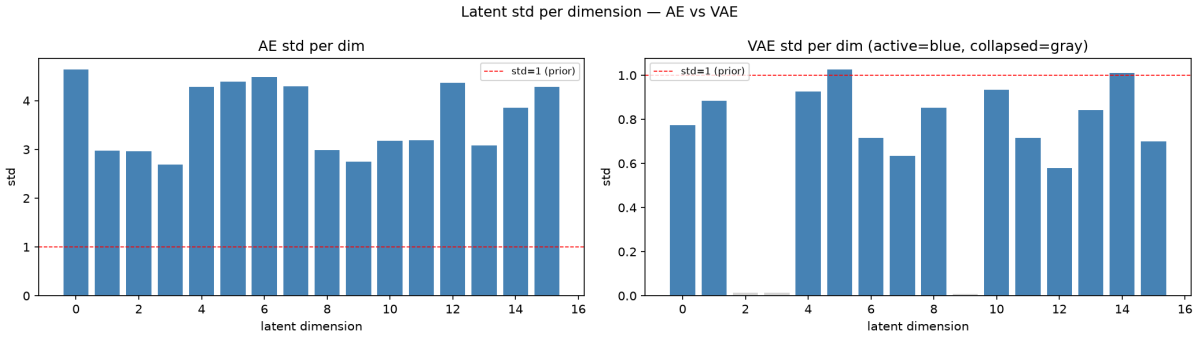


Figure 14: Standard deviation per latent dimension for the AE (left) and VAE (right). Grey bars in the VAE panel indicate collapsed dimensions; the red dashed line marks $\sigma = 1$ (unit Gaussian prior).

6.3 2D projections

Figures 15 and 16 show t-SNE and UMAP projections of the balanced 10,000-point sample coloured by ground-truth churn label.

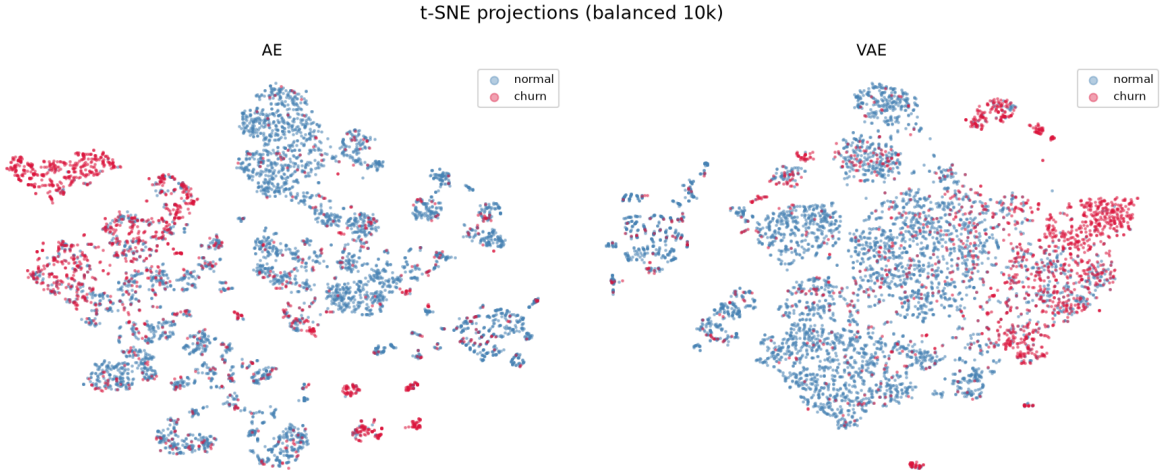


Figure 15: t-SNE projections (balanced 10k). Normal customers in blue, churners in red. Both models show partial but incomplete class separation.

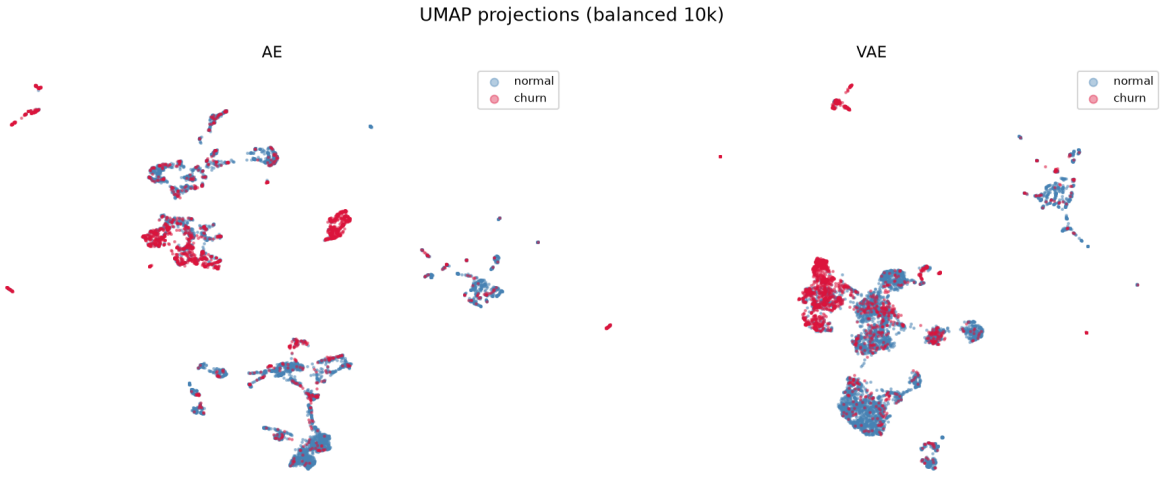


Figure 16: UMAP projections (balanced 10k). Churn customers form distinct peripheral clusters outside the main normal-customer manifold in both models.

Figure 17 overlays the VAE ELBO anomaly score (`VAE_recon_kl`) on the VAE UMAP, confirming that high-anomaly regions align spatially with the churn cluster.

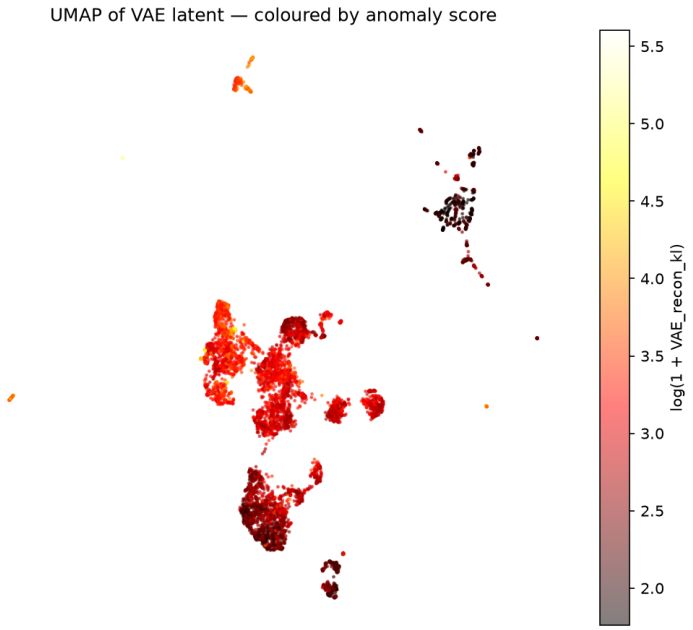


Figure 17: VAE UMAP coloured by $\log(1 + \texttt{VAE_recon_kl})$. High anomaly scores concentrate in the same peripheral regions occupied by churners.

6.4 Class separability

Table 9 reports the Silhouette score (on standardised latent vectors) and linear probe AUC for both models.

Table 9: Latent space separability on the 20k stratified test sample.

Model	Dims	Silhouette (std)	Linear probe AUC
AE (all dims)	16	**0.252**	**0.848**
VAE (all dims)	16	0.156	0.847
VAE (active dims only)	13	0.152	0.846

6.5 Discussion

The AE achieves a higher Silhouette score (0.252 vs. 0.156), indicating more natural geometric separation between churn and normal customers. The VAE's KL regularisation pulls all codes toward the prior, reducing inter-class distance and lowering the Silhouette score. Despite this geometric difference, both models yield nearly identical linear probe AUC ($\approx$0.848), which is competitive with the best supervised model from Workshop 1 (Random Forest AUC = 0.863).

This finding reveals an important property of these representations: even though the autoencoders were trained entirely without labels, their internal codes carry enough churn signal for a linear classifier to approach supervised performance. The representations are not organised by class the way a supervised encoder would be, but the unsupervised manifold structure still captures the statistical irregularity of churner behaviour. Restricting the VAE to its 13 active dimensions produces no meaningful improvement in either metric, confirming that the 3 collapsed dimensions carry negligible information rather than useful redundancy.

7 Analysis

The AE outperforms the VAE on reconstruction quality (MSE 1.8$\times$ lower, near-perfect categorical accuracy) because its deterministic bottleneck preserves input detail without KL pressure. The VAE trades fidelity for a smoother latent space, a cost that is visible on heterogeneous tabular data.

For anomaly detection, the VAE ELBO score (`VAE_recon_kl`, AUC = 0.812) approaches the supervised baselines (best AUC = 0.863) without any label supervision. The remaining $\sim$5-point gap reflects the fundamental difficulty of defining the anomaly class by reconstruction error alone rather than by explicit labels: precision and recall stay well below the supervised models (F1 0.42 vs. 0.58).

The latent space analysis confirms that both representations encode churn signal: a linear probe reaches AUC $\approx$ 0.848 on the raw codes. However, Silhouette scores remain modest (AE 0.25, VAE 0.16), indicating the signal is present but not geometrically concentrated — explaining why probe AUC is high while 2D projections show only partial class separation.

Overall, autoencoders provide useful unsupervised signal that approaches supervised performance in AUC, but do not replace labelled classifiers where precision and recall matter.

References

[1] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

[2] Rizvi, A. (2018). *WSDM – KKBox's Churn Prediction Challenge*. Medium. Retrieved from `https://asad-99rizvi.medium.com/wsdm-kkboxs-churn-prediction-challenge-d7c9cfa21cbd`